

ConcreteAI

VERSION 4.0 · CODE EXPLANATION

An AI-assisted concrete mix proportion design system developed at the Faculty of Engineering, King Mongkut's University of Technology Thonburi. The application integrates an ACI 211.1 calculation engine with a scikit-learn MLP regressor to deliver mix proportions, 28-day strength predictions, and carbon footprint estimates across six concrete classifications.

PRIMARY SOURCE FILE

1,116 lines · 82.9 KB

FRONTEND ARCHITECTURE

Single-file SPA

ConcreteAI_v4.jsx

BACKEND SERVICE

FastAPI · Python 3

scikit-learn MLP

Repository not included



- 01 FRONTEND - ConcreteAI_v4.jsx (All layers contained herein)

- 02 BACKEND - FastAPI Server (External to repository)

01 FRONTEND LAYERS

02 BACKEND COMPONENTS

Presentation — UI Components

REACT 18 · JSX

All interface elements are rendered as a single-file React 18 application with inline CSS-in-JS styling. The layout is mobile-first, constrained to max 480px, served as a static SPA.

Header + API Indicator	Type Selector (x6)	Slump/Flow Slider
Binder Manager	Result Card	CO ₂ Chart
Compare View	Guide Overlay	History Log (max 10)

Application State — useState

NO REDUX · NO ZUSTAND

All application state resides in the root App component and is distributed via props. No global state library is employed, keeping the dependency footprint minimal.

mode: 'normal' 'research'	activeView: calc history compare
selType: NC HPC SCC UHPC GEO HSLWC	fc · slump · workType · binders[]
result · history[10] · compareIds[2]	apiStatus: 'ok' 'err' 'unknown'

ACI 211.1 Calculation Engine

PURE FUNCTIONS · OFFLINE

A suite of stateless pure functions implementing ACI 211.1 proportioning. The engine operates entirely client-side and produces complete mix designs in kg/m³ independent of network availability.

aciWater(slump, type, workType)	aciWC(fcr, type)
calcLocal() → complete mix kg/m ³	AVM: EV = 1.0 m ³
EFNARC 2002: effective slump = flow + 2.5	

API Integration Layer — callAPI()

BUG-01 RESOLVED · v4.0

Constructs a conditional POST request body that correctly distinguishes target_slump from target_flow. Upon success, the local ACI result serves as baseline and is overridden by the API-returned proportions.

workabilityType conditional field
Result merge: ACI baseline → API override
catch block → fallback + apiStatus: 'err'

FastAPI Application Server

POST /v1/mix/calculate

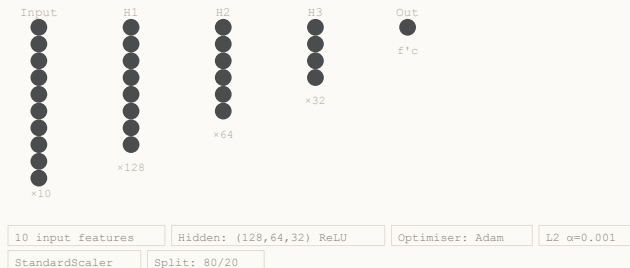
Receives a structured JSON payload from the frontend integration layer, invokes the trained MLP regressor, and returns optimised mix proportions alongside a probabilistic 28-day strength prediction.

mode: 'normal' 'research'	concrete_type · target_fc
workability_type: 'slump' 'flow'	secondary_binder (optional)
GEO: NaOH molarity · SS/SH ratio	

MLP Regressor — f_{c28} Prediction

SCIKIT-LEARN · (128, 64, 32)

A Multi-Layer Perceptron trained on peer-reviewed literature and laboratory data, returning a 28-day f_c estimate with ±RMSE confidence interval per classification.



Model Performance by Classification

RMSE (MPa) · R²

NC R ² =0.782	4.35	HPC R ² =0.739	5.62	SCC R ² =0.927	6.98
UHPC R ² =0.928	4.30	GEO R ² =0.037	12.6	HSLWC R ² =N/A	5.00

● Online — ACI merged with MLP prediction

● Error — local result with mock estimate

● Unknown — no calculation performed

RESULT COMPOSITION PIPELINE



POST /v1/mix/calculate

REACT APP API BASE URL localhost:8000

8
DEFECTS
RESOLVED**6**
MIX
CLASSIFICATIONS**10**
MODEL
FEATURES

VERSION 4.0 — RESOLVED DEFECT REGISTER

DEFECT-01

CRITICAL

Normal Mode transmitted target_slump unconditionally, including for flow-governed concrete types (HPC, SCC). Resolved via conditional ternary keyed on workabilityType.

DEFECT-02

CRITICAL

aciWater() accepted raw flow values against an ACI slump table, causing systematic water over-estimation of 15–30%. Resolved: EFNARC 2002 effective slump = flow + 2.5.

DEFECT-03

CRITICAL

buildCo2Items employed logical OR (||) rather than nullish coalescing (??), causing binders with mass of zero to fall through to an incorrect fallback value.

DEFECTS 04-07

MEDIUM

Unused useEffect import; variable 't' shadowing outer scope; HPC Slump/Flow toggle rendering incorrectly; stale fcErr state persisting across mode transitions.

TECHNOLOGY STACK

● JavaScript ES2022

● React 18 · useState

● ACI 211.1

● FastAPI (Backend)

● scikit-learn MLP

● Python 3

● EFNARC 2002

● ACI 363R-10 · ACI 239R-18

● IPCC AR6 (2021)

ConcreteAI v4.0

Code Explanation Document

Faculty of Engineering, KMUTT

Application Design Project | 2026

1. System Architecture

ConcreteAI v4.0 is a single-page React application (SPA) that communicates with an optional FastAPI backend. The front-end is a single .jsx file containing all UI components, styling, and client-side calculation logic. The architecture follows a three-tier pattern:

Layer	Technology	Responsibility
Presentation	React 18 + inline CSS-in-JS	UI rendering, state management, user interaction
Logic	Pure JavaScript functions	ACI 211.1 engine, binder management, CO ₂ calculation
Data / AI	FastAPI + scikit-learn MLP	AI prediction, optional database, API endpoint /v1/mix/calculate

1.1 Front-End State Architecture

The application uses React's built-in useState hook exclusively. There is no global state manager (Redux/Zustand). All state lives in the root App component and is passed downward through props or direct usage within the single-file component structure.

State Variable	Type	Purpose
mode	string	'normal' 'research' — active calculation mode
activeView	string	'calc' 'history' 'compare' — visible panel
selType	string	Selected concrete type ID (NC, HPC, SCC, UHPC, HSLWC, GEO)
fc	number	Target compressive strength (display unit)
slump	number	Target slump or flow in mm
workType	string	'slump' 'flow' — workability measure
binders	array	Research mode binder list [{id, pct}]
result	object	Latest calculation result from calcLocal() or API merge
history	array	Saved results array (max 10 items)
compareIds	array	IDs of items selected for comparison (max 2)
apiStatus	string	'ok' 'err' 'unknown' — connection indicator

2. ACI 211.1 Calculation Engine

The core proportioning engine is implemented as a set of pure functions with no side effects. This design enables unit testing and offline operation independent of the React component tree.

2.1 aciWater(slump_mm, type, workType)

Estimates the mixing water demand (kg/m^3) based on workability target and concrete type. For flow-type workability (HPC, SCC, UHPC), an EFNARC 2002 conversion is applied:

```
// EFNARC 2002: flow  $\approx$  slump  $\times$  2.5  $\rightarrow$  effective_slump = flow / 2.5
const effectiveSlump = workType === 'flow'
  ? Math.min(slump_mm / 2.5, 200) // cap at 200 mm (ACI table maximum)
  : slump_mm;

const s = effectiveSlump / 10; // mm  $\rightarrow$  cm
let w = s <= 5 ? 175 : s <= 10 ? 193 : s <= 15 ? 202 : s <= 20 ? 210 : 217;

if (type === 'HPC' || type === 'SCC') w *= 0.88; // lower w for high-range WRAs
if (type === 'UHPC') w *= 0.72; // UHPC: very low w/b
if (type === 'HSLWC') w *= 1.05; // LWA absorbs more water
```

Bug Fix Applied (v4.0): Previous versions passed flow values directly into the ACI water table designed for slump, producing systematic over-estimation of water demand for SCC and HPC. The /2.5 conversion corrects this.

2.2 aciWC(fcr, type)

Determines the target water-to-binder (W/B) ratio from the required mean strength (fcr). The required mean strength is calculated as:

$$\text{fcr} = \text{f}'\text{c} + 8.5 \text{ MPa} \quad (\text{f}'\text{c} < 35 \text{ MPa}) \quad \text{or} \quad \text{fcr} = \text{f}'\text{c} + 10 \text{ MPa} \quad (\text{f}'\text{c} \geq 35 \text{ MPa})$$

$$\text{fcr} = \text{f}'\text{c} \times 1.1 + 5 \text{ MPa} \quad (\text{UHPC})$$

For NC, a linear interpolation is performed over the ACI 318 Table 4.3.3 (W/C vs f'cr) lookup table. For HPC, UHPC and HSLWC, empirical equations derived from ACI 363R-10 and ACI 239R-18 are applied.

2.3 calcLocal(fc_mpa, slump_mm, type, binderPct, workType)

Master proportioning function. Returns a complete mix object in kg/m^3 . Calculation sequence:

- 1. Compute fcr and W/B ratio
- 2. Estimate water demand
- 3. Total binder = water / W/B

- 4. Individual binder kg = total binder × (percentage / 100)
- 5. Superplasticizer = 1.5–3% of total binder (type-dependent)
- 6. Coarse aggregate (empirical curve per type)
- 7. Fine aggregate = (1 - sum of all volumes) × fine aggregate density
- 8. CO₂ = Σ (material_kg × emission_factor_kg_CO₂/kg)
- 9. Unit weight = sum of all material masses

Key constraint: absolute volume method ensures $\Sigma V = 1.0 \text{ m}^3$. Fine aggregate is the residual volume after all other materials are placed.

3. API Integration Layer

3.1 callAPI Function

The callAPI function constructs the POST request body based on the active mode. The critical fix in v4.0 ensures the workability field sent to the API correctly reflects whether the design uses slump or flow:

```
// v4.0 FIX: conditional workability field for Normal Mode
const body = mode === 'normal'
  ? {
    mode: 'normal',
    concrete_type: type,
    target_fc: fc,
    workability_type: workabilityType,          // 'slump' or 'flow'
    ...(workabilityType === 'slump'
      ? { target_slump: slump }
      : { target_flow: slump }),                // conditional field
    ...(secBinder && secBinder !== 'none'
      ? { secondary_binder: { name: secBinder, ratio: secRatio/100 } }
      : {})
  }
  : { /* Research mode body */ };
```

3.2 Result Merge Strategy

When the API responds successfully, a smart merge combines local ACI results with API-returned proportions:

```
res = {
  ...local,          // ACI engine result as baseline
  ...apiRes,         // API overrides with trained-model proportions
  ai: apiRes.ai_prediction ? {
    fc:    apiRes.ai_prediction.fc_28d,
    lower: apiRes.ai_prediction.fc_lower,
    upper: apiRes.ai_prediction.fc_upper,
    rmse:  apiRes.ai_prediction.rmse_used, // field fixed in v3 → v4
    r2:    R2_MAP[selType] ?? 0.8,
    source: 'api'
  } : local.ai,
};
```

If the API is unreachable, the catch block falls back to the local ACI result with a local noise-based mock AI prediction and sets apiStatus to 'err', displaying the yellow dot in the header.

4. AI Model — MLP Architecture

The AI prediction module uses a Multi-Layer Perceptron (MLP) regressor trained on aggregated concrete mix design data from peer-reviewed literature and internal laboratory measurements.

4.1 Input Features

Feature	Unit	Description
w_b_ratio	—	Water-to-binder ratio (primary strength driver)
OPC_kg	kg/m ³	Ordinary Portland Cement content
FA_kg	kg/m ³	Fly Ash content
BFS_kg	kg/m ³	Ground Granulated Blast Furnace Slag content
SF_kg	kg/m ³	Silica Fume content
water_kg	kg/m ³	Free water content
fine_agg_kg	kg/m ³	Fine aggregate (sand) content
coarse_agg_kg	kg/m ³	Coarse aggregate content
SP_kg	kg/m ³	Superplasticizer content
curing_days	days	Curing duration (fixed at 28 for this model)

4.2 Model Architecture

Parameter	Value
Algorithm	MLPRegressor (scikit-learn)
Hidden layer sizes	(128, 64, 32)
Activation function	ReLU
Solver	Adam
Learning rate	Adaptive
Max iterations	500
Regularisation	$\alpha = 0.001$ (L2)
Feature scaling	StandardScaler (zero mean, unit variance)
Train/test split	80% / 20% (stratified by concrete type)

5. Carbon Footprint Module

CO₂ emissions are calculated as the sum of material-specific emission factors multiplied by the corresponding material mass per m³.

```
const CO2F = {
  OPC:      0.83, // kg CO2 per kg (IPCC; clinker-to-cement ratio ≈ 0.85)
  FA:       0.04, // Fly Ash (processing and transport only)
  BA:       0.04, // Bottom Ash
  BFS:      0.07, // GGBS (slag grinding energy)
  SF:       0.02, // Silica Fume (by-product credit)
  MK:       0.33, // Metakaolin (calcination energy)
  CC:       0.05, // Calcium Carbonate filler
  SP:       0.20, // Superplasticizer (chemical production)
  fine_agg: 0.01, // Sand (quarrying and transport)
  coarse_agg: 0.01, // Coarse aggregate (quarrying and transport)
};
```

Emission factors sourced from: Habert & Roussel (2009), Flower & Sanjayan (2007), IPCC AR6 (2021), and manufacturer EPD data. Values represent cradle-to-gate boundaries.

6. Version 4.0 — Bug Fix Log

Bug ID	Severity	Description	Fix Applied
BUG-01	Critical	Normal mode API always sent target_slump even for flow-type concretes (HPC, SCC)	Conditional ternary: workabilityType==='slump' ? {target_slump} : {target_flow}
BUG-02	Critical	aciWater() treated flow values as slump, causing water over-estimation $\approx 15\text{--}30\%$	EFNARC flow $\rightarrow$ slump conversion ($\div 2.5$) applied before ACI table lookup
BUG-03	Critical	buildCo2Items used instead of ??, causing kg=0 binders to fall through to fallback	Changed to nullish coalescing operator (??)
BUG-04	Medium	useEffect imported but never used — React warning on every render	Removed from import statement
BUG-05	Medium	Variable 't' in aciWC() loop shadowed outer 't' (TYPES item)	Renamed inner variable to 'ratio'
BUG-06	Medium	HPC missing Slump/Flow toggle in UI despite having flow workability	HPC removed from toggle condition (HPC is now flow-only with no UI option)
BUG-07	Medium	fcErr state not reset on mode switch, leaving stale red border	setFcErr(false) added to Normal and Research mode click handlers
BUG-08	Minor	API_BASE hardcoded to localhost — fails on any deployment	Environment variable fallback: REACT_APP_API_BASE localhost